

JC4004 – Computational Intelligence

Sessions 35 & 36: Structured state-space models

Dr Jari Korhonen (jari.korhonen@abdn.ac.uk)

Dr Yongchao Huang (Yongchao.huang@abdn.ac.uk)

Dr Shahzad Mumtaz (shahzad.mumtaz@abdn.ac.uk)

Oct-Dec 2025

What are structured state-space models?

- *State-space models* (SSM) have been traditionally used as mathematical models for physical systems
 - State-space models are characterised by a set of inputs, outputs, and *state variables* related by first-order differential equations
 - It has been suggested that SSMs can model the “memory cells” involved in memory functions in brain
- *Structured state-space models for sequences (S4)* model very long-term dependencies more efficiently than transformers
 - S4 reduces the complexity of classic state-space models, allowing them to be used in neural networks

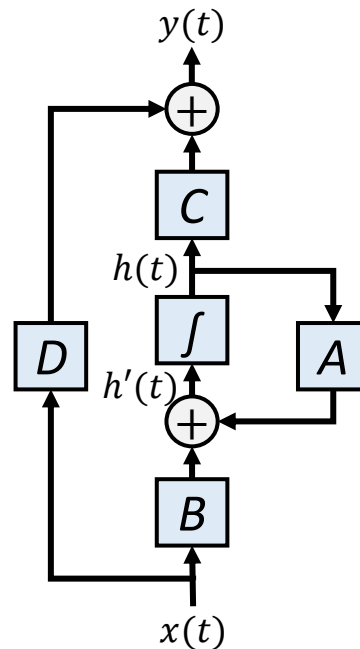
SSM definition

- Classic state-space models are defined by mapping input signal $x(t)$ into a latent presentation $h(t)$ and then to output signal $y(t)$

$$h'(t) = \mathbf{A}h(t) + \mathbf{B}x(t); \quad h(t) = \int h'(t)dt$$

$$y(t) = \mathbf{C}h(t) + \mathbf{D}x(t)$$

- Here, $x(t)$ and $y(t)$ are 1D vectors, $h(t)$ is a N -dimensional latent representation (hidden variable)
- $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$, and $\mathbf{D}$ are learnable weight matrices



Discretising SSM

- In the original model, $x(t)$ and $y(t)$ are continuous functions of time
- Assuming discrete input x_k , i.e., sampled version $x_k = x(k\Delta)$ where Δ is the size of a time step, we can discretise the state matrices as:

$$\begin{aligned}\bar{A} &= I + \Delta A \\ \bar{B} &= \Delta B \\ \bar{C} &= C\end{aligned}$$

- Note that we omit state matrix D in the discrete version

Proof for discretising SSM

- For small Δ , we can approximate $h'(t) = [h(t) - h(t - \Delta)]/\Delta$
- Letting $h_k = h(k\Delta)$, and $h_{k-1} = h(k\Delta - \Delta)$:

$$\begin{aligned} h'(t) &\approx h_k - h_{k-1} = \Delta(\mathbf{A}h_{k-1} + \mathbf{B}x_k) \\ \Rightarrow h_k &= (\mathbf{I} + \Delta\mathbf{A})h_{k-1} + \Delta\mathbf{B}x_k \end{aligned}$$

- Discretising is straightforward, but we need to decide Δ

Discrete SSM

- Discrete version of the model maps sequence x_k to sequence y_k :

$$\begin{aligned}h_k &= \bar{\mathbf{A}}h_{k-1} + \bar{\mathbf{B}}x_k \\ y_k &= \bar{\mathbf{C}}h_k\end{aligned}$$

- Here, h_k is a discrete-time hidden space: the equation looks similar with vanilla RNN, but the underlying assumptions are different!
- Considering full sequences, the equations would look like:

$$\begin{aligned}h_k &= \bar{\mathbf{A}}^k \bar{\mathbf{B}}x_0 + \bar{\mathbf{A}}^{k-1} \bar{\mathbf{B}}x_1 + \dots + \bar{\mathbf{B}}x_k \\ y_k &= \bar{\mathbf{C}}h_k = \bar{\mathbf{C}}\bar{\mathbf{A}}^k \bar{\mathbf{B}}x_0 + \bar{\mathbf{C}}\bar{\mathbf{A}}^{k-1} \bar{\mathbf{B}}x_1 + \dots + \bar{\mathbf{C}}\bar{\mathbf{B}}x_k\end{aligned}$$

Discrete SSM matrix computations

- Graphical presentation of SSM matrix computations shown here:

$$\begin{array}{c} h_k \\ \text{[4 yellow squares]} \end{array} = \begin{array}{c} \bar{A} \\ \text{[4x4 green grid]} \end{array} \begin{array}{c} h_{k-1} \\ \text{[4 yellow squares]} \end{array} + \begin{array}{c} \bar{B} \\ \text{[4 blue squares]} \end{array} \cdot \begin{array}{c} x_k \\ \text{[1 red square]} \end{array}$$

$$\begin{array}{c} y_k \\ \text{[1 pink square]} \end{array} = \begin{array}{c} C \\ \text{[1x4 grey grid]} \end{array} \cdot \begin{array}{c} h_k \\ \text{[4 yellow squares]} \end{array}$$

SSM complexity

- In basic SSM, computing all the k hidden states for a sequence would require a huge number of matrix multiplications
 - Not computationally feasible, except for very short sequences!

$$h_k = \underbrace{\bar{A} \cdot \bar{A} \cdot \dots \cdot \bar{A}}_k \cdot \bar{B} \cdot x_0 + \dots + \bar{B} \cdot x_k$$

Computing outputs in S4

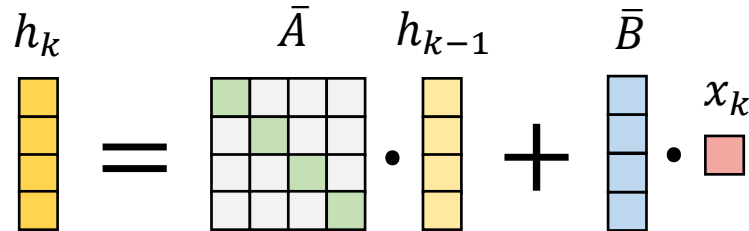
- We could express the complete representation as $y = \mathbf{K} * x$, where operation $*$ is the traditional convolution:

$$y = \mathbf{K} * x \Rightarrow y_k = \sum_{i=0}^{\infty} K_i x_{k-i}$$

- Matrix $\mathbf{K}$ can be computed when $\bar{A}$, $\bar{B}$, and C are known
 - From x and $\mathbf{K}$, y can be computed e.g., via *fast Fourier transform (FFT)*
 - However, this is computationally feasible for very short sequences only
- *Structured SSM for sequences (S4)* aims to make computational complexity more reasonable

Structured SSM (S4)

- Structured SSM (S4) imposes a special structure to matrix $\bar{A}$, reducing computational complexity significantly
 - A commonly used structure is a *diagonal matrix*, when multiplication of two $N \times N$ matrices is reduced from n^3 to n multiplications

$$h_k = \bar{A} h_{k-1} + \bar{B} x_k$$


HiPPO projection

- *High order polynomial projection operators (HiPPO)* is a framework for online compression of sequences
 - Given input sequence c , we can define HiPPO projection for generating hidden state sequence f as:

$$c_{k+1} = \left(1 - \frac{A}{k}\right) c_k + \frac{1}{k} B f_k, \text{ where:}$$

$$A_{n,k} = \begin{cases} (2n+1)^{1/2}(2k+1)^{1/2}, & \text{if } n > k \\ n+1, & \text{if } n = k \\ 0, & \text{if } n < k \end{cases}$$

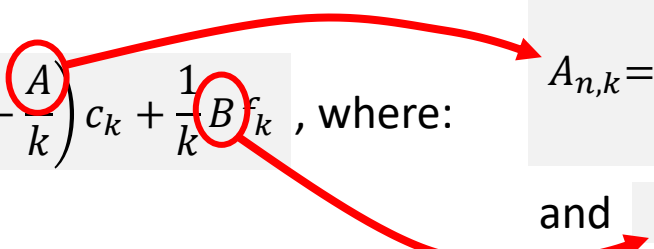
$$\text{and } B_n = (2n+1)^{1/2}$$

HiPPO projection

- *High order polynomial projection operators (HiPPO)* is a framework for online compression of sequences
 - Given input sequence c , we can define HiPPO projection for generating hidden state sequence f as:

$$c_{k+1} = \left(1 - \frac{A}{k}\right) c_k + \frac{1}{k} B f_k, \text{ where:}$$
$$A_{n,k} = \begin{cases} (2n+1)^{1/2} (2k+1)^{1/2}, & \text{if } n > k \\ n+1, & \text{if } n = k \\ 0, & \text{if } n < k \end{cases}$$

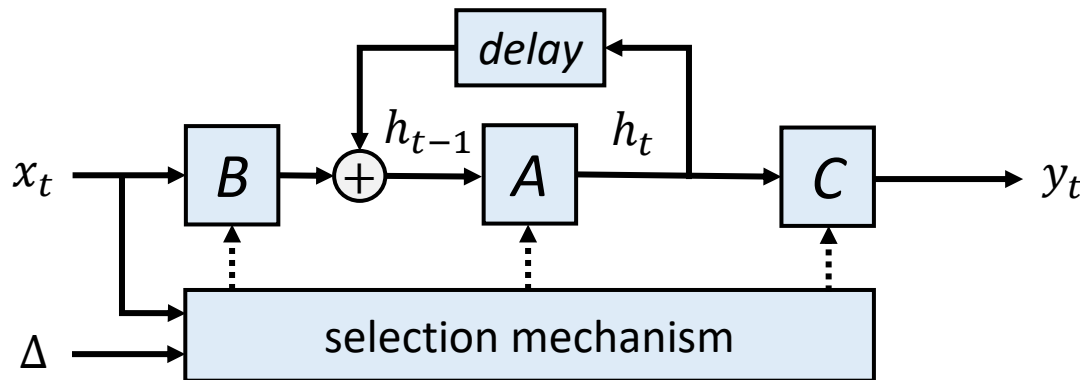
and $B_n = (2n+1)^{1/2}$



- Note that we only need to compute A and B once!

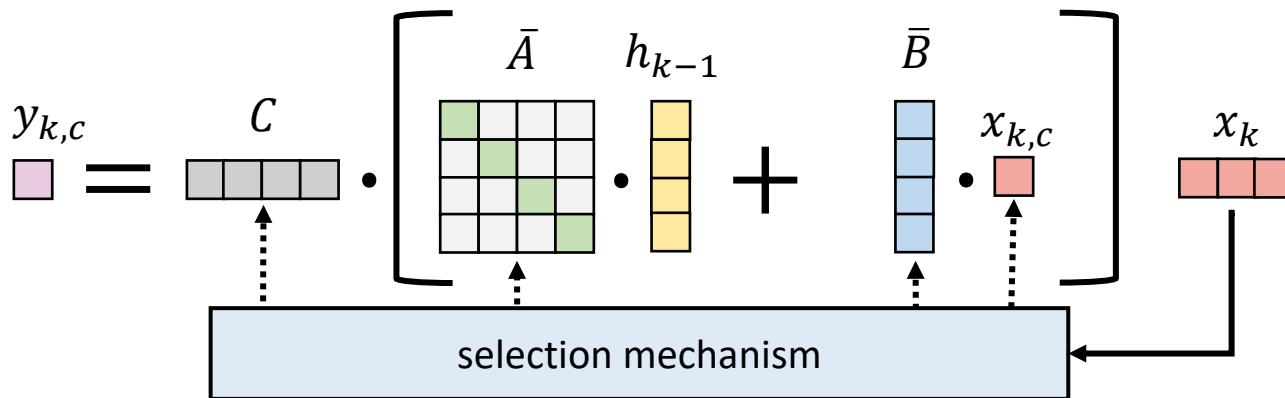
Mamba model architecture

- In *Mamba* architecture, SSM layer is defined by four parameters: step size Δ , weight matrix A , biases B , and C : A and B discretised as $\bar{A}$ and $\bar{B}$
 - Matrix A initialized using a method inspired by the HiPPO framework



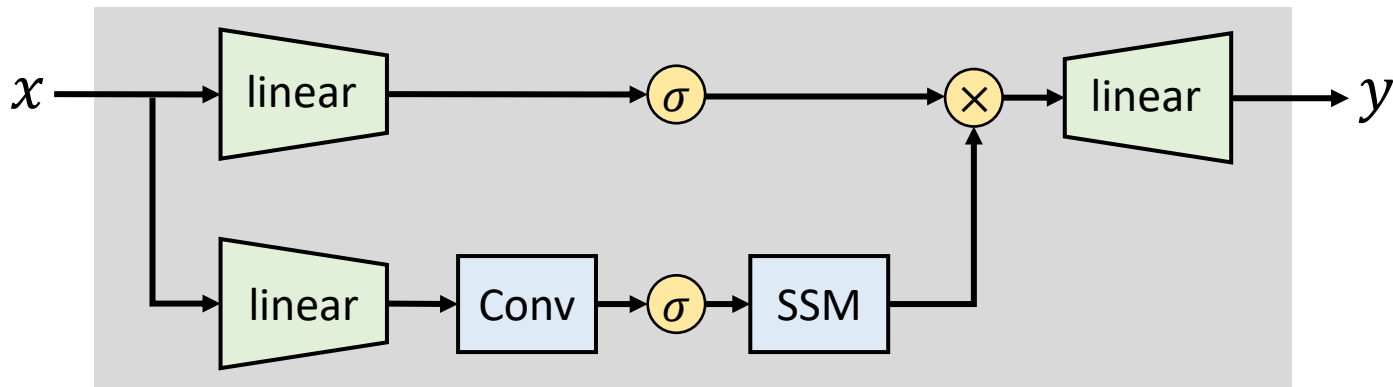
Mamba selection mechanism

- In Mamba, $\bar{A}$, $\bar{B}$, C , and Δ are not fixed, but selected dynamically as a function of input x_k
 - Hidden state partially constructed at each iteration by computing each channel of y_k from input channels and x_k independently



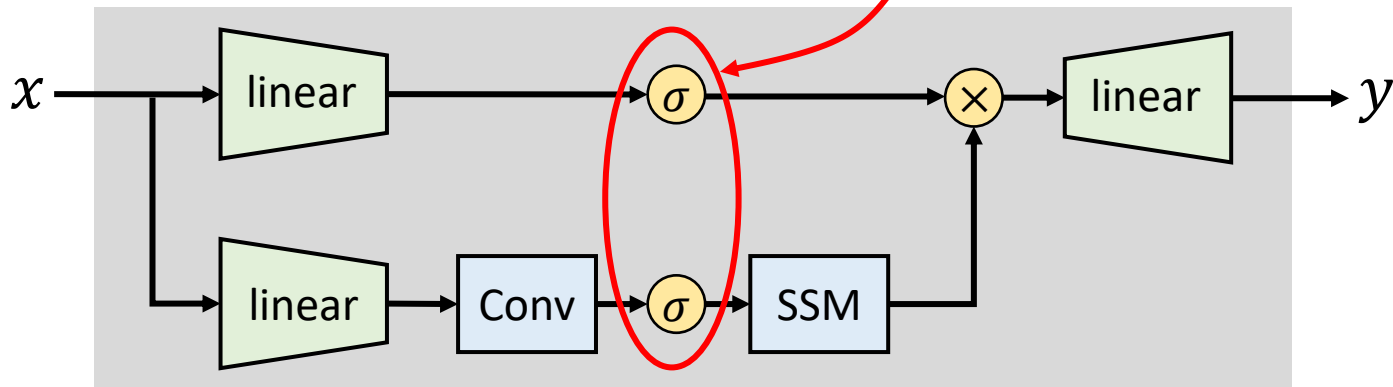
Mamba block

- Mamba blocks can be used as layers in neural networks, just like RNNs or transformers
 - Basic Mamba block includes linear projections and a gating mechanism in addition to selective SSM transformation



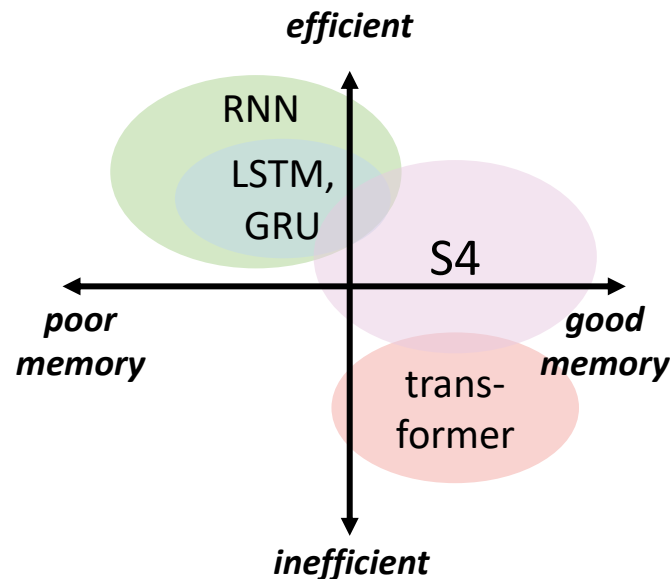
Mamba block

- Mamba blocks can be used as layers in neural networks, just like RNNs or transformers
 - Basic Mamba block includes linear projections and a gating mechanism in addition to selective SSM



Comparison of S4 with other models

- Mamba can be used for similar tasks as RNNs and transformers
 - S4 models (e.g., Mamba) have better memorising potential than RNNs, and more efficient computation and memory usage than transformers
 - Mamba reports to produce state-of-the-art results in some tasks, but still lags behind transformers in other tasks



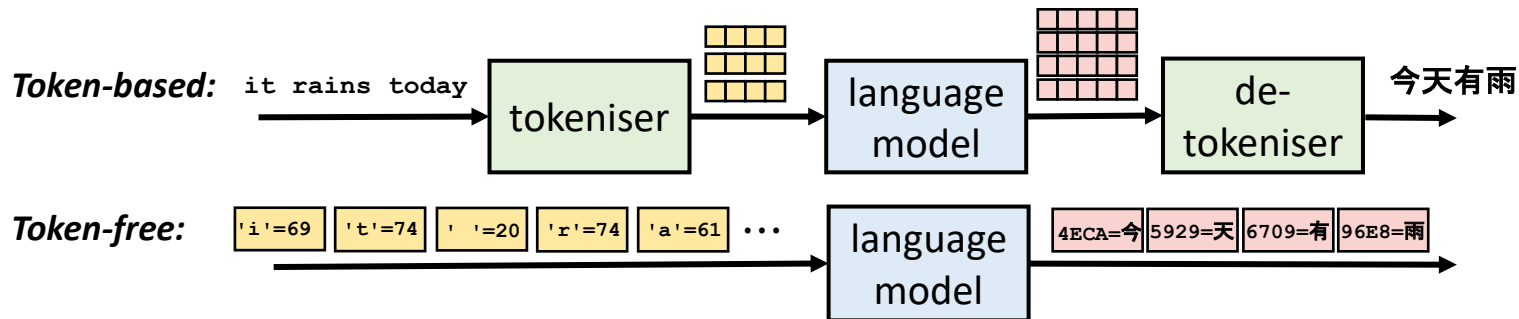
Break: any questions?

Mamba applications

- Since Mamba is a sequence model, it has been experimented on several sequence-based tasks, especially in NLP
 - Most Mamba architectures resemble transformer architectures, where transformer blocks are replaced with Mamba blocks
- As for transformers, Mamba blocks have also been adopted for a wider range of tasks, including vision
 - For example, Vision and Video Mamba for computer vision
- We will walk through some of the promising Mamba architectures

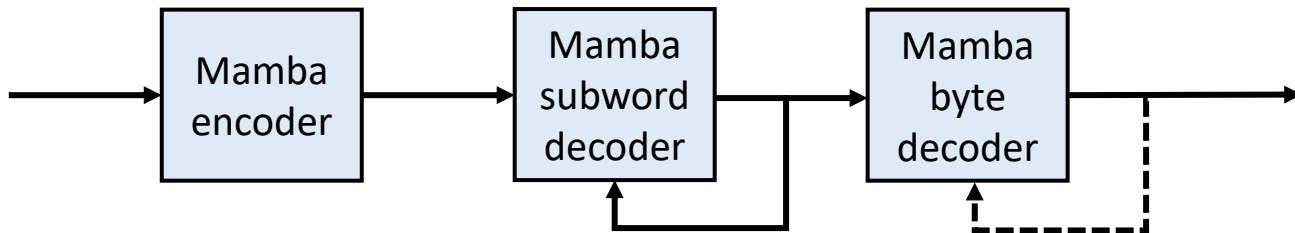
MambaByte

- Most NLP models use tokens for words or subwords
 - Allows shorter sequences, but has limitations in speed and versatility
- SSMs can handle long sequences easier than transformers
 - *MambaByte* was designed for *token-free* language tasks: characters are fed to the model directly as a stream of bytes



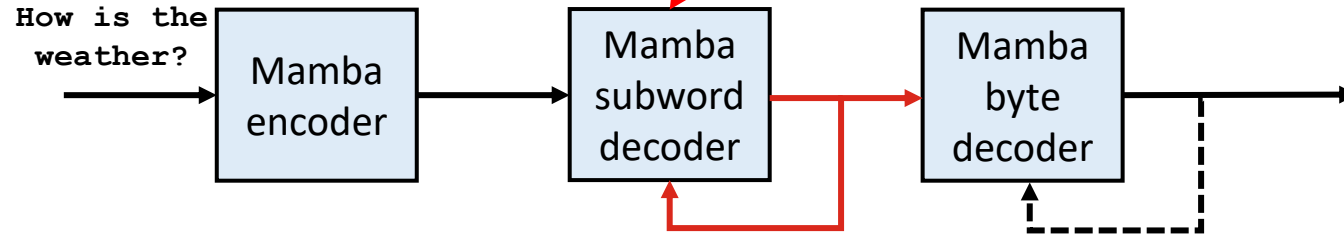
MambaByte speculative decoding

- MambaByte model uses two decoders: smaller subword level decoder and larger byte level decoder
 - Smaller subword decoder produces subword drafts that are verified by larger byte decoder and corrected if necessary
 - If the subword prediction does not match with byte prediction top-3, the byte decoder corrects the word in autoregressive mode



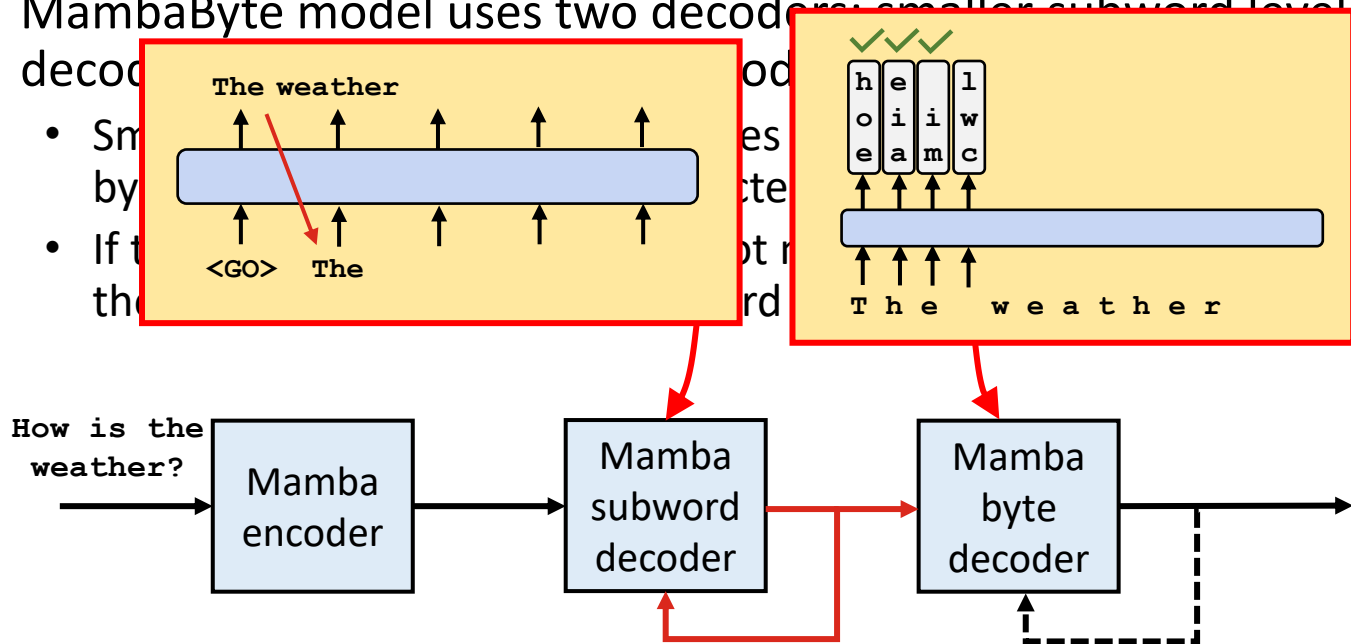
MambaByte speculative decoding

- MambaByte model uses two decoders: smaller subword level decoder
 - Smaller subword level decoder generates subword drafts that are verified by the byte decoder if necessary
 - If the subword draft does not match with byte prediction top-3, the draft is rejected and the model continues in autoregressive mode



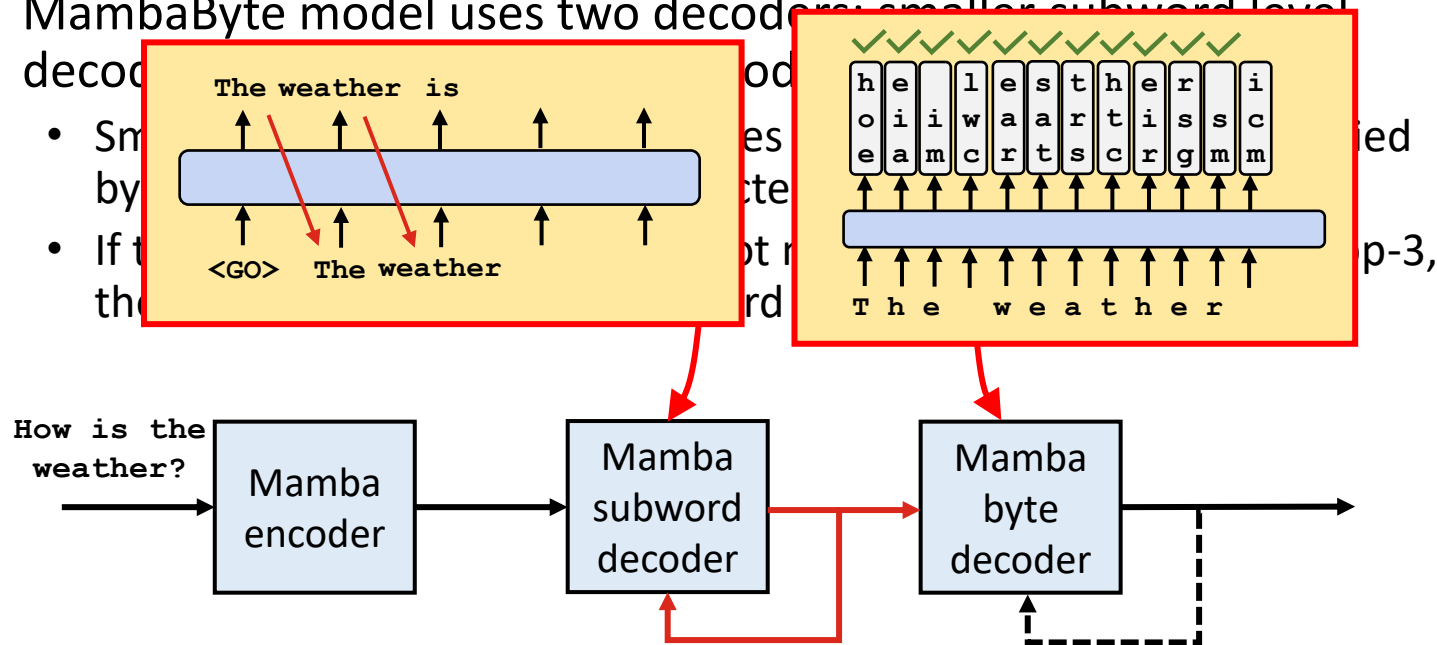
MambaByte speculative decoding

- MambaByte model uses two decoders: smaller subword level decoder and larger byte level decoder
- Small subword decoder generates subword candidates
- If the subword decoder generates a candidate that is not a valid word, the byte decoder is used to generate a byte-level candidate



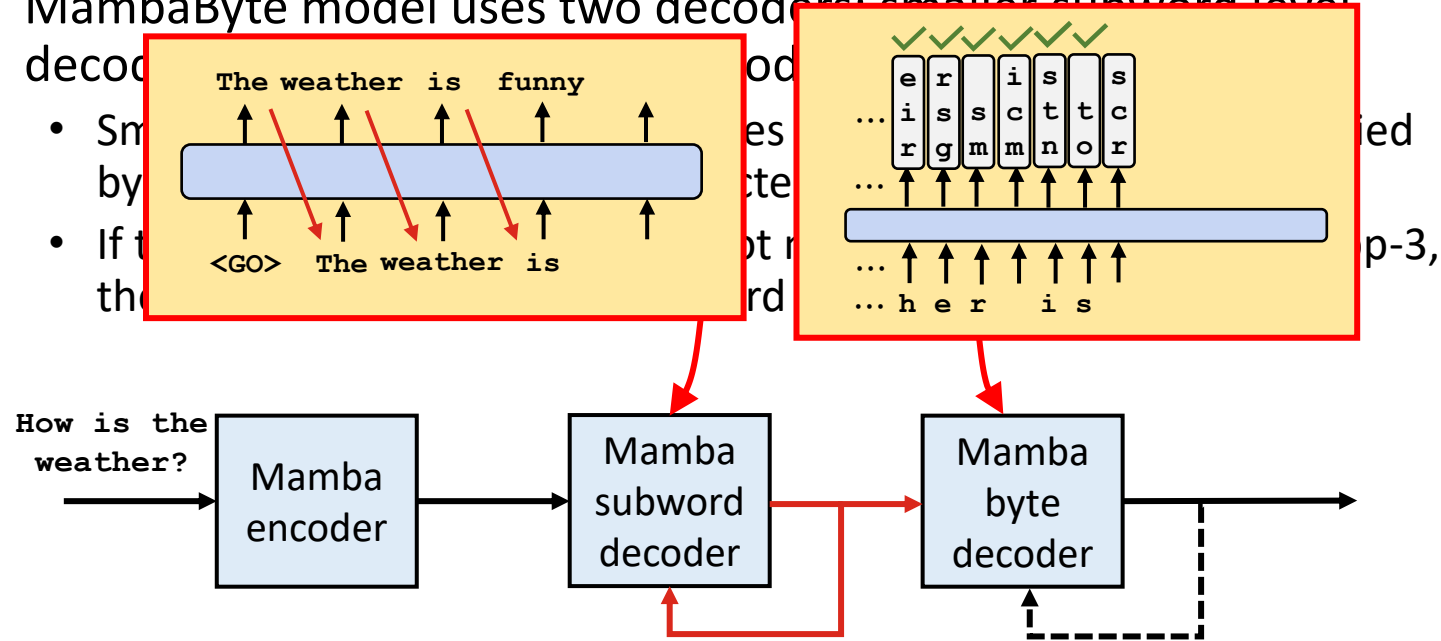
MambaByte speculative decoding

- MambaByte model uses two decoders: a smaller subword level decoder and a larger byte level decoder.
- Small subword decoder
- If the subword decoder predicts a word, the byte decoder is not needed.
- If the subword decoder predicts a subword, the byte decoder is needed to decode the subword into bytes.



MambaByte speculative decoding

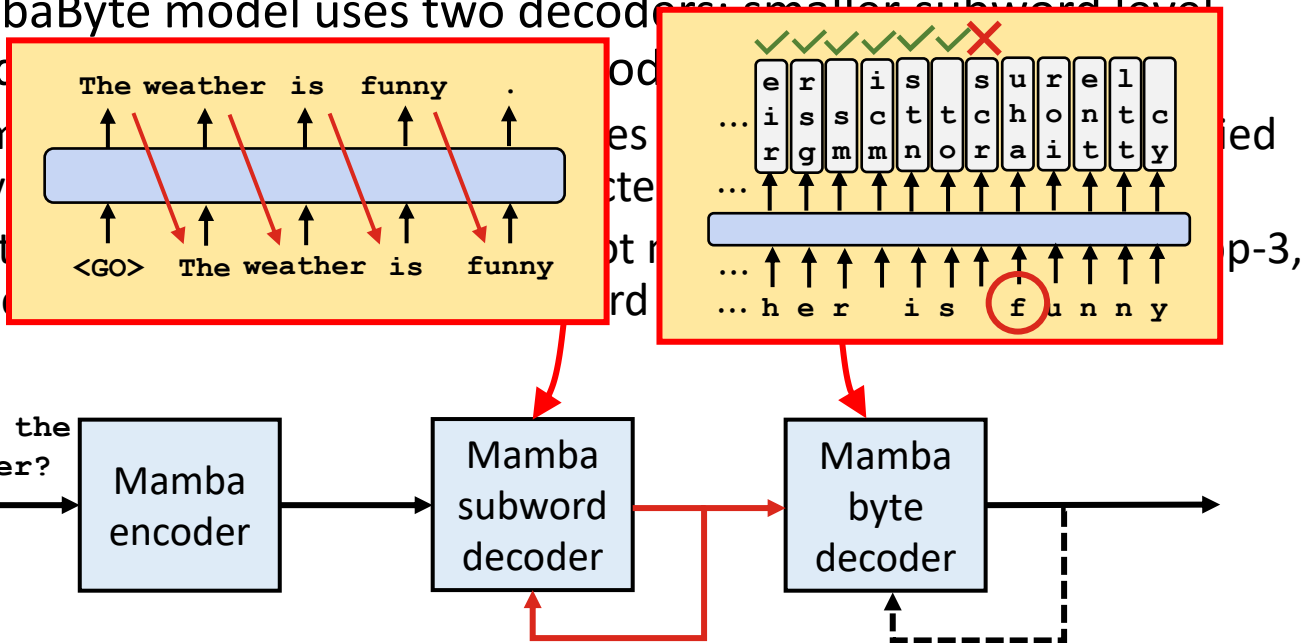
- MambaByte model uses two decoders: a smaller subword level decoder and a larger byte level decoder.
- Small subword decoder generates subword hypotheses.
- If the subword hypotheses are correct, the byte decoder can be used to generate the final output.



MambaByte speculative decoding

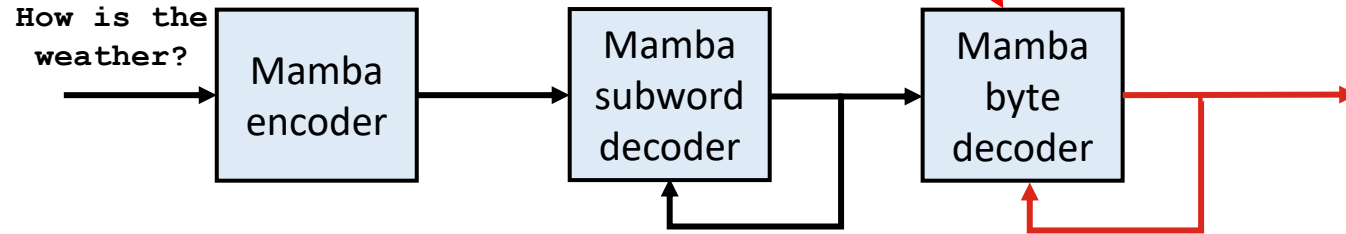
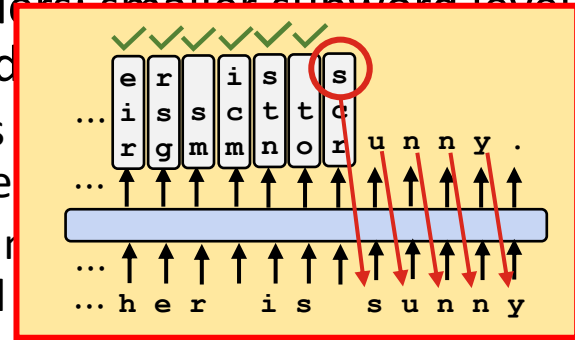
- MambaByte model uses two decoders: a smaller subword-level decoder and a larger byte-level decoder.

- Small by
- If the



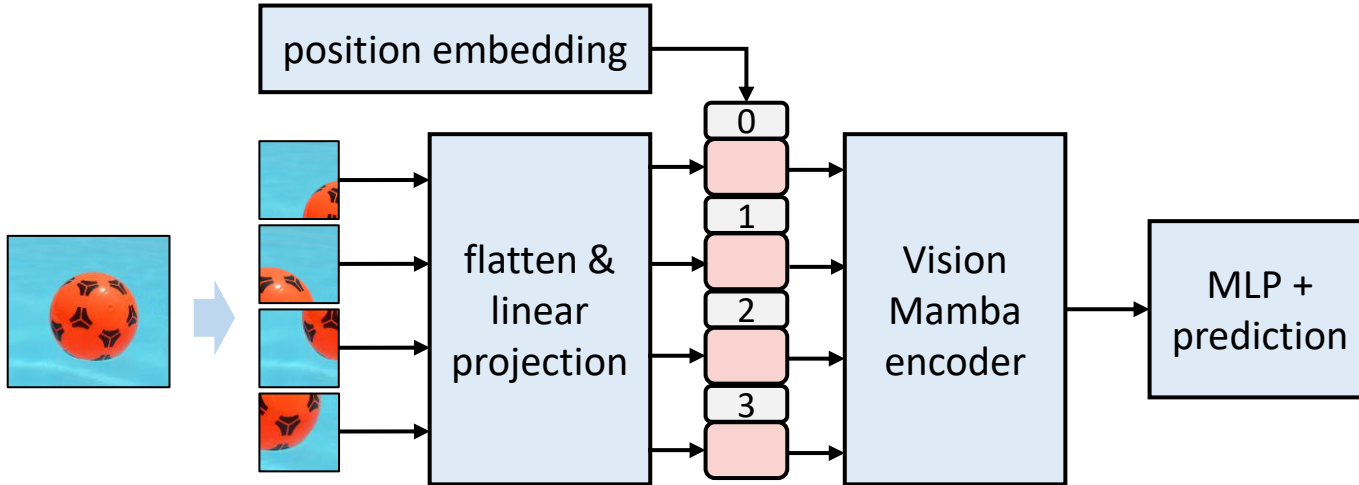
MambaByte speculative decoding

- MambaByte model uses two decoders: smaller subword level decoder and larger byte level decoder
 - Smaller subword decoder produces predictions by larger byte decoder and corrects errors
 - If the subword prediction does not match the byte decoder's output, the byte decoder corrects the word



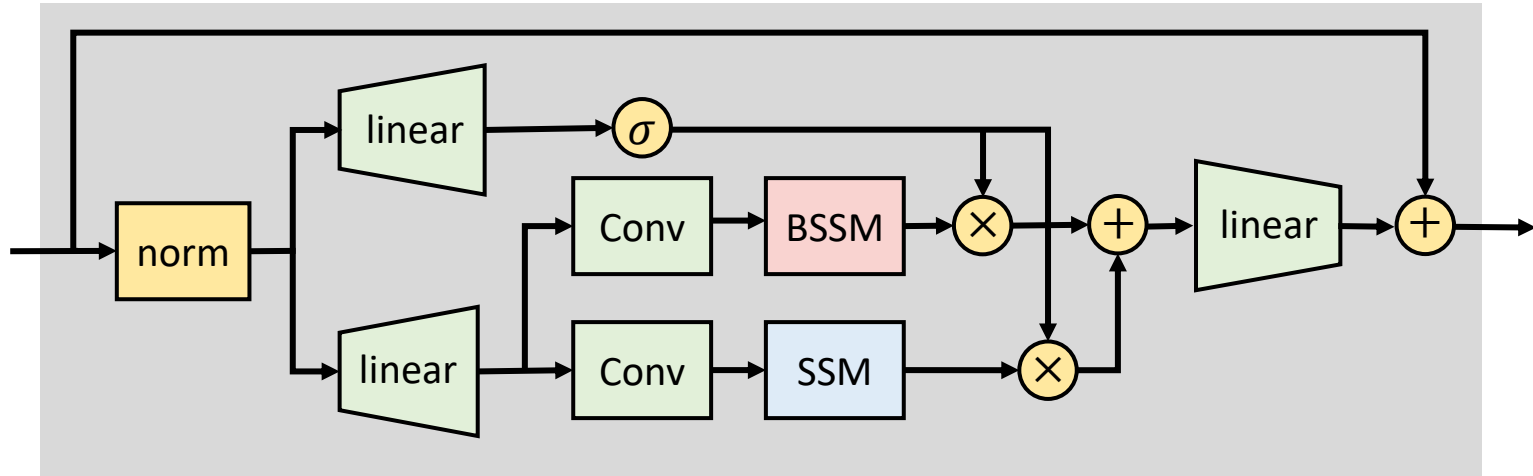
Vision Mamba (ViM)

- Vision Mamba (ViM)* architecture is heavily inspired by ViT, but using bidirectional Mamba blocks instead of transformer blocks



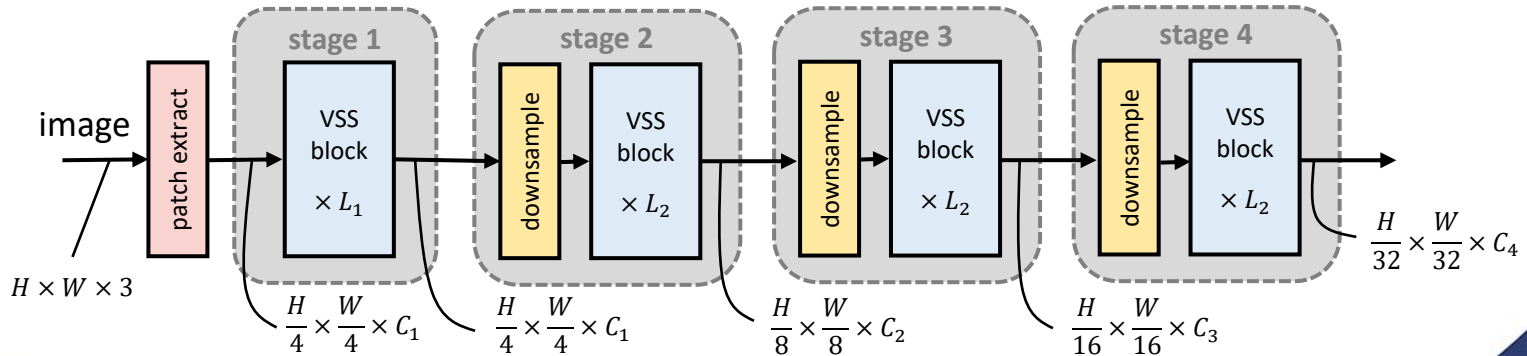
Vision Mamba block

- Vision Mamba block is a variant of basic Mamba block, with two SSM layers: one forwards and one backwards



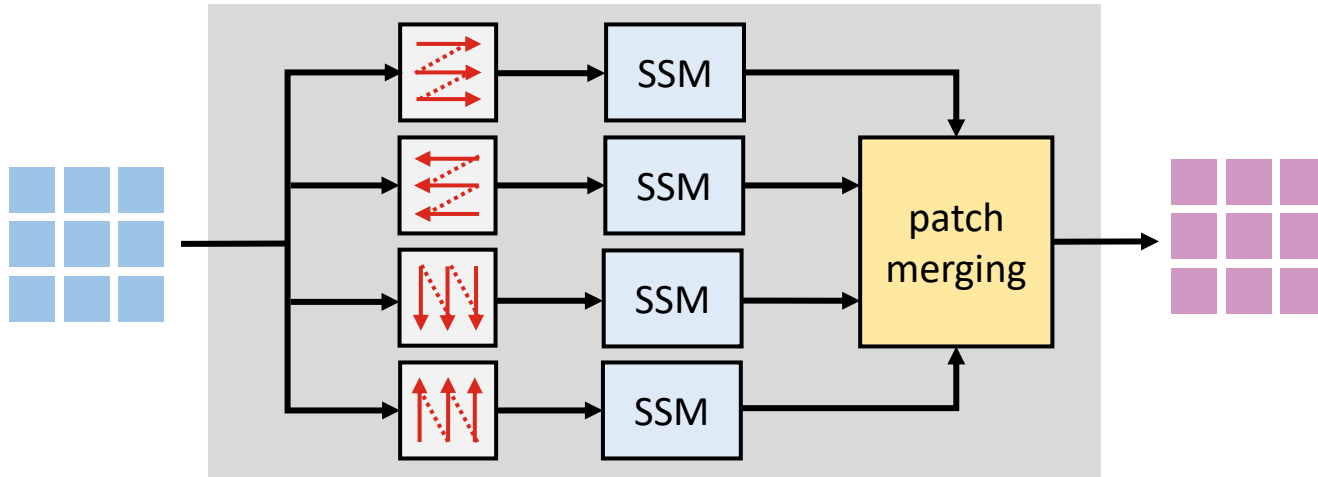
VMamba

- *VMamba* is another proposed Mamba-based architecture for vision tasks
 - Differs from ViM by processing images in hierarchical manner with downsampling between the stages of *visual state space (VSS)* blocks
 - Also introduces 2D selective scan (SS2D) for scanning the batches



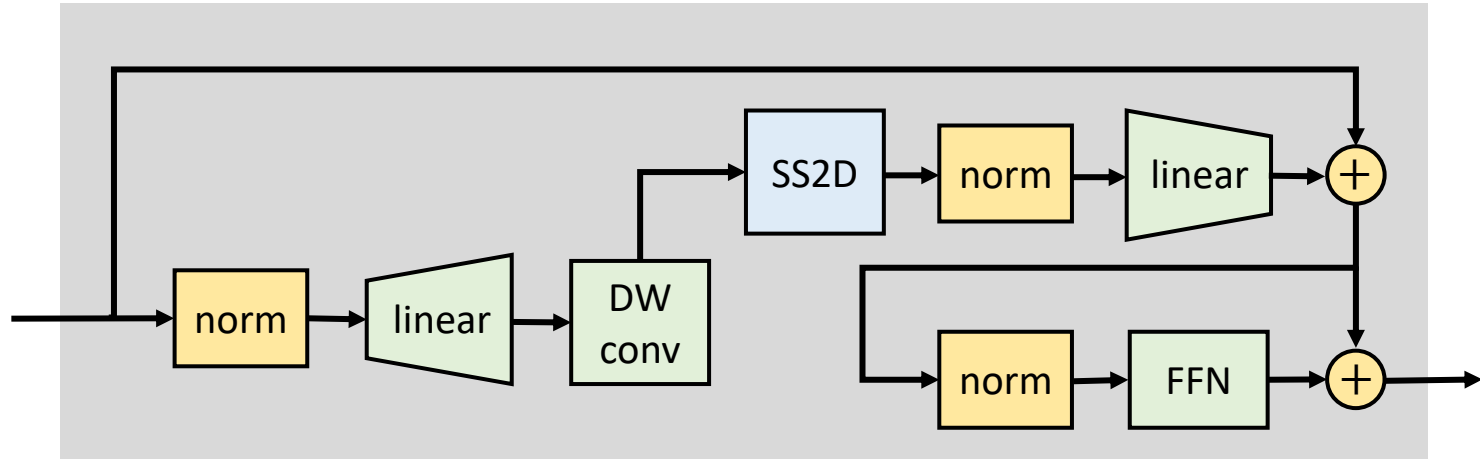
VMamba SS2D layer

- VMamba blocks use SS2D layers with four branches of SSM with different scanning orders for the branches



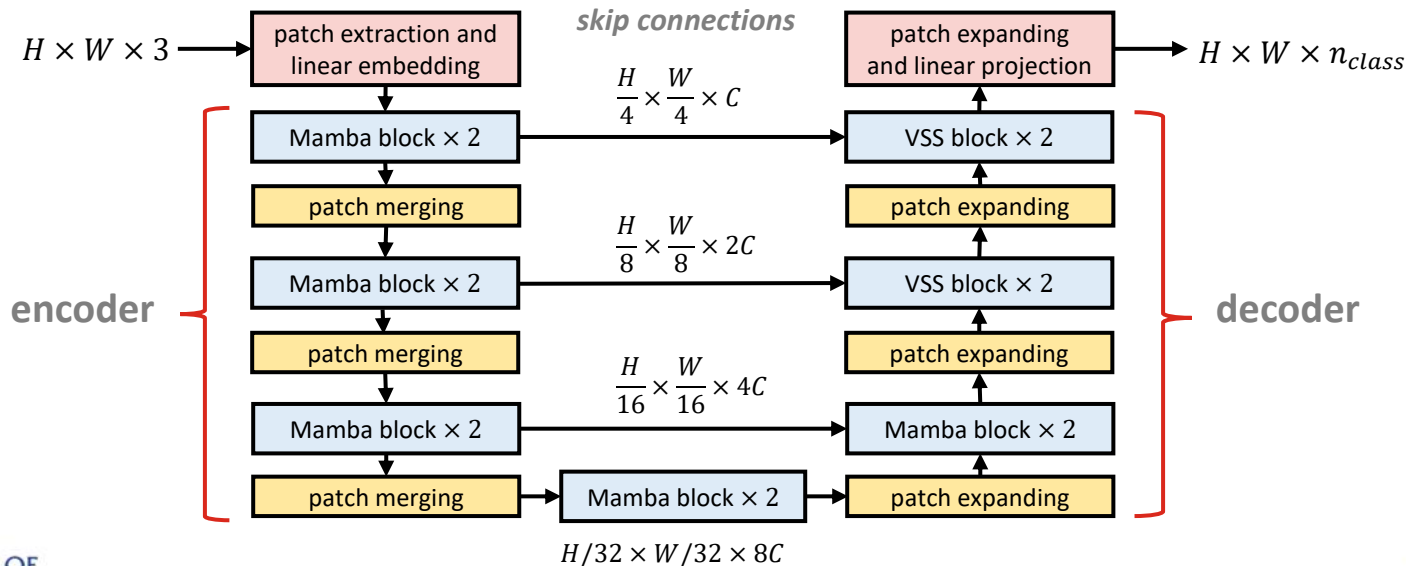
VMamba block

- VMamba block is another variant of Mamba block, with SS2D layer as the core component



Mamba UNets

- Mamba UNets resembling Swin-T-UNet with Mamba blocks instead of transformers have been proposed for image segmentation



Notes about Mamba for vision

- In addition to ViM and Vmamba, a number of other Mamba-based architectures for computer vision have also been proposed
 - A lot of research is currently ongoing on Mamba and S4s in general
 - Most of the proposed architectures are based on transformer architectures, where transformer blocks are replaced with Mamba blocks
- It is not yet sure if vision SSMs will constitute a credible alternative to transformers, but the preliminary results look promising
 - Most of the computer vision applications are not obviously sequence-based, so the usefulness of SSM for vision tasks is debated

Summary

- *State-space models (SSM)* have traditionally been used as mathematical models in physics and control engineering
 - Baseline SSM uses continuous signals: discretising required for discrete signals, e.g., sequence-to-sequence transformations
 - Complexity of baseline SSMs is prohibitive for neural networks, but use of structured state matrices can reduce complexity essentially
- Mamba architecture is based on *structured SSMs for sequences (S4)*
 - Reportedly, Mamba models memorise long-term dependencies in the input signal almost as well as transformers, with lower complexity
 - Several models have been derived from Mamba, e.g., MambaByte for token-free language tasks, and Vision Mamba for computer vision

References

- Gu et al.: *“HiPPO: Recurrent Memory with Optimal Polynomial Projections,”* NIPS 2020.
- Gu, Goel, Re: *“Efficiently Modeling Long Sequences with Structured State Spaces,”* ICLR 2022.
- Gu & Dao: *“Mamba: Linear-Time Sequence Modeling with Selective State Spaces,”* <https://arxiv.org/abs/2312.00752>, 2023.
- Wang et al.: *“MambaByte: Token-free Selective State Space Model,”* COLM 2024.
- Zhu et al.: *“Vision Mamba: Efficient Visual Representation Learning with Bidirectional State Space Model,”* ICML 2024.
- Liu et al.: *“Vmamba: Visual State Space Model,”* <https://arxiv.org/pdf/2401.10166>, 2024.
- <https://medium.com/@ulalaparis/the-mamba-effect-mamba-models-gaining-ground-f2d2c9b9245c>

Questions and discussion